SEMESTER 2/20252026

SPECIAL TOPIC IN DATA ENGINEERING (SECP3843)

SECTION 01

TUTORIAL APACHE SPARK

LECTURER:

DR ARYATI BINTI BAKRI

NO	NAME	MATRIC NO
1	LIM YU HAN	A23CS0241
2	LING YU QIAN	A23CS0301
3	TEH RU QIAN	A23CS0191

1. Business Understanding

1.1 Project Overview

The purpose of this project is to process the educational census data from the Brazilian Government Open Data Portal by building an ETL (Extract, Transform, Load) pipeline using Apache Spark. The raw CSV data includes information about schools, structural, ethnical, geographical, economic and more collected between 2010 to 2020.

The project converts large and unstructured data into a structured Star Schema stored in PostgreSQL, allowing for business intelligence (BI) analysis and dashboard creation.

1.2 Business Problem

The raw data presents some business challenges:

- Large file sizes (about 2.2GB total)
- Around 370 columns per dataset
- Slow processing using traditional tools such as Pandas
- Difficulty generating dashboards directly from raw data

1.3 Business Objectives

- Build a scalable ETL pipeline using Apache Spark
- Convert raw data into Parquet format
- Design a Star Schema data warehouse model
- Store transactional data in PostgreSQL
- Support BI dashboards and analytical queries

2. Data Understanding

The data source is from the Brazilian National Basic Education Census (Censo Escolar), publicly available through the government's open data portal.

2.1 Characteristics

- **Datatype**: CSV files
- **Time Period**: 2010-2020
- **Number of Columns**: ~370 columns
- **Total Dataset Size**: ~2.2GB
- **Estimated Records**: ~3 million rows

2.2 Relational Data Model

The raw data is restructured into a Star Schema to support analytics and BI reporting.

Fact Table: fact_censo_escolar

It links to dimension tables to provide context for each metric and supports historical analysis of enrollments and teachers from 2010 to 2020.

Column Name	Description
QT_DOC_BAS	Number of Teachers in the basic education (TOTAL)
QT_DOC_INF	Number of Teachers in the basic education (child education)

QT_DOC_FUND	Number of Teachers in the basic education (elementary education)
QT_DOC_MED	Number of Teachers in the basic education (high school)
QT_MAT_BAS	Number of enrollments in the basic education (TOTAL)
QT_MAT_INF	Number of enrollments in the basic education (child education)
QT_MAT_FUND	Number of enrollments in the basic education (elementary education)
QT_MAT_MED	Number of enrollments in the basic education (high school)
QT_MAT_BAS_ND	Number of enrollments in the basic education - Skin color/Race Not Declared
QT_MAT_BAS_BRANCA	Number of enrollments in the basic education - Skin color/Race Branco
QT_MAT_BAS_PRETA	Number of enrollments in the basic education - Skin color/Race Preto
QT_MAT_BAS_PARDA	Number of enrollments in the basic education - Skin color/Race Parda
QT_MAT_BAS_AMARELA	Number of enrollments in the basic education - Skin color/Race Amarela
QT_MAT_BAS_INDIGENA	Number of enrollments in the basic education - Skin color/Race Indígena
NU_ANO_CENSO	Census' year

Dimension Table

dim_local

Column Name	Description
NO_UF	State's name
SG_UF	State's abbreviation
CO_UF	State's code
NO_MUNICIPIO	City's name
CO_MUNICIPIO	City's code

dim_tp_localizacao

Column Name	Description

TP_LOCALIZACAO	The school location (urban rural)
id	Primary key

dim_tp_dependencia

Column Name	Description
TP_DEPENDENCIA	The school administration (state, city, private)
id	Primary key

dim_in_agua_potavel

Column Name	Description
IN_AGUA_POTAVEL	has access to drinkable water
id	Primary key

dim_in_energia_inexistente

Column Name	Description
IN_ENERGIA_INEXISTENTE	has (NOT) access to energy
id	Primary key

dim_in_esgoto_inexistente

Column Name	Description
IN_ESGOTO_INEXISTENTE	has (NOT) access to energy
id	Primary key

dim_in_equip_nenhum

Column Name	Description
IN_EQUIP_NENHUM	no electronic equipment
id	Primary key

dim_in_internet

Column Name	Description
IN_INTERNET	has internet
id	Primary key

dim_in_computador

Column Name	Description
IN_COMPUTADOR	has computer
id	Primary key

dim_in_refeitorio

Column Name	Description
IN_REFEITORIO	has canteen
id	Primary key

dim_in_biblioteca

Column Name	Description
IN_BIBLIOTECA	has library
id	Primary key

dim_in_banheiro

Column Name	Description
IN_BANHEIRO	has restroom
id	Primary key

3. Data Preparation

First, the pipeline sets up its path variables to avoid environment errors. Since Apache Spark can have some compatibility issues when run on a Window machine, the following is the explicit definition of the Hadoop folder path at the top of the script.

```
import os
os.environ["HADOOP_HOME"] = "C:\\hadoop"
os.environ["PATH"] = os.environ["PATH"] + ";C:\\hadoop\\bin"
```

Then, to avoid running out of resources, the Spark session is customized with a different amount of memory. A particular driver jar file path is also provided to enable Spark to communicate with local database subsequently.

```
spark = SparkSession.builder \
        .appName("CensoEscolarStarSchema") \
        .config("spark.sql.shuffle.partitions", "4") \
        .config("spark.driver.memory", "10g") \
        .config("spark.executor.memory", "6g")\
        .config("spark.master", "local[4]") \
        .config("spark.jars", "C:\\spark\\jars\\postgresql-42.7.11.jar") \
        .getOrCreate()

print(spark)
```

Once the pipeline is set up, it uses Python's glob library to locate each of the individual yearly CSV files within the download directory. Then , Spark reads all these files together into a single dataframe, using the right semicolon delimiter and text encoding, without being forced to use slow schema auto-detect.

```
csv_files = glob.glob("C:/Users/User/Downloads/Y3S2/special topic

print(f"Found {len(csv_files)} files:")  # Should print 12
print(csv_files)

data_csv = (
    spark
    .read
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "false")
    .option("delimiter", ";")
    .option("encoding", "iso-8859-1")
    .load(csv_files)
)
```

Lastly, to achieve low operational read times to query flat text source structures that are heavy, the multi-file data configuration is compressed and stored directly in a local Parquet directory storage path in the column-oriented format.

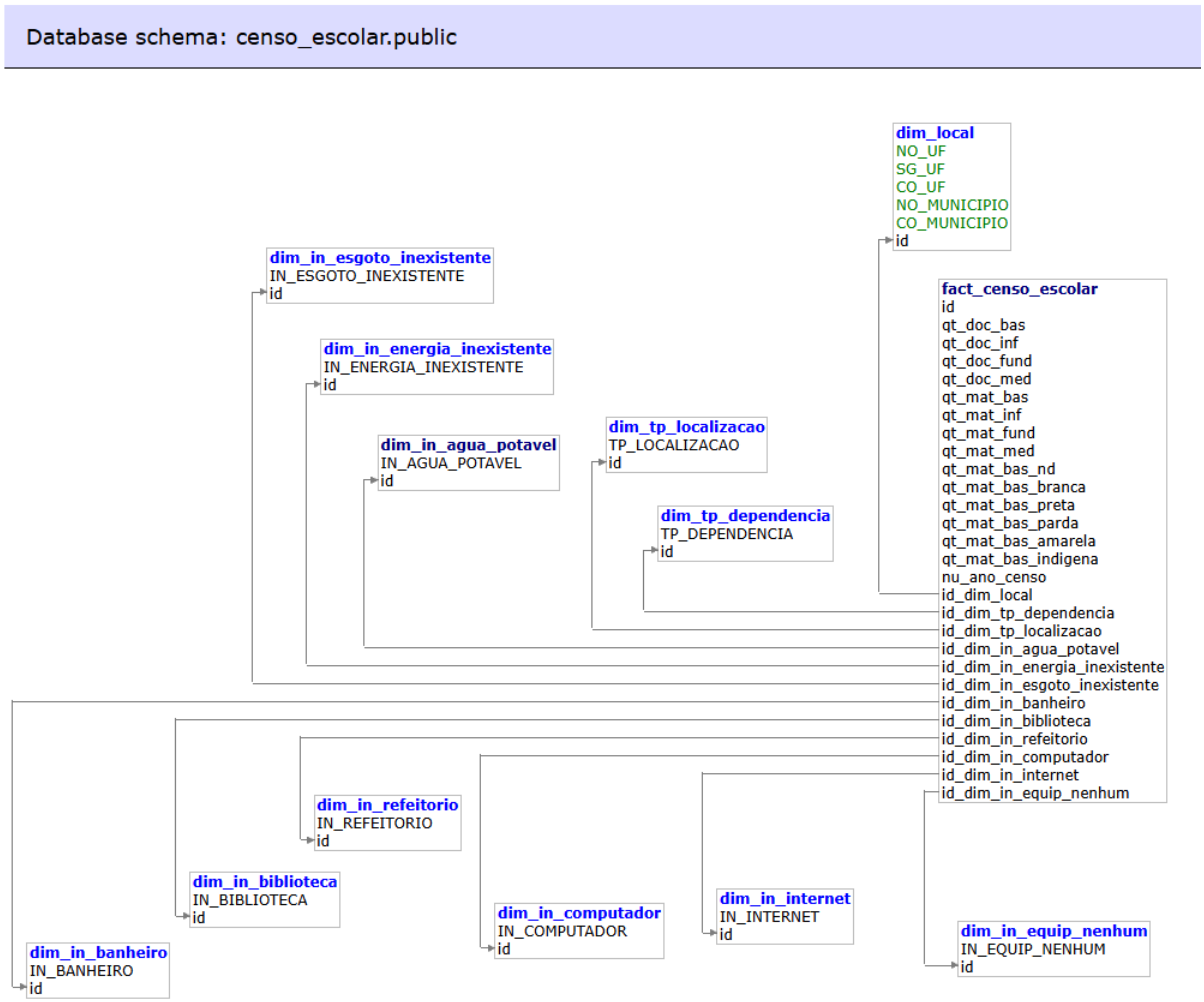
```
data_csv.write.parquet("./data/censo_escolar.parquet")
```

The data can be compressed to a large extent by storing it as a Parquet file. This physical stage breaks the rows into physical binary blocks. Parquet file enables new extraction joins to fetch explicit tracking categories in a quick and complete manner without having to traverse slow uncompressed scans.

4.0 Modeling

4.1 Star Schema Architectural Design

The source structure is unnormalized and is optimized to a Multi-Dimensional Star Schema to establish high performing OLAP query structures. In this configuration, certain categories of metadata are stored in twelve separate dimension tables, connected directly to a central fact table with typical relational foreign key relationships.



4.2 Dimension Tables Configuration

The dimension tables are designed to contain certain categorical data values, which are distinct from the main metrics. First, DIM_LOCAL is created to cluster regional columns, such as the names of the states and the municipal numbers. Then, a loop is executed to dynamically build each of the individual lookup dimensions for the school infrastructure columns (11 columns).

```
INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",
    "TP_LOCALIZACAO",
    "IN_AGUA_POTAVEL",
    "IN_ENERGIA_INEXISTENTE",
    "IN_ESGOTO_INEXISTENTE",
    "IN_BANHEIRO",
    "IN_BIBLIOTECA",
    "IN_REFEITORIO",
    "IN_COMPUTADOR",
    "IN_INTERNET",
    "IN_EQUIP_NENHUM"
]
```

The code uses .distinct() to remove duplicate rows from the main Parquet file and adds an incremental identifier row to serve as the primary key for each dimension to ensure proper extraction.

```
for table_name, table_config in DIMENSION_TABLES_CONFIG.items():

    print(f"[{datetime.now()}] Writing {table_name}")

    data\
    .select(
        [
            F
            .col(field["field"])
            .cast(field["type"])
            .alias(field["field"])

            for field
            in table_config["fields"]
        ]
    )\
    .distinct()\
    .withColumn(
        "id", F.monotonically_increasing_id()
    )\
    .write\
    .jdbc(
        **POSTGRES_CONFIG,
        table=table_name,
        mode="overwrite"
    )
```

Each dimension table is loaded directly into the PostgreSQL target database with an overwrite option to get a clean start.

4.3 Centralize Fact Table Architecture

After securely saving the twelve dimensions in the database, the pipeline creates the main fact tables, FACT_CENSO_ESCOLAR. This table contains all of the numerical school metrics and reference surrogate keys that reference the dimension layouts. For example, core staffing volumes, enrollment metrics and relational reference keys.

Spark performs multiple left joins to correctly map the fact table information with the raw data columns and the dimension information generated. Once the keys have been matched, the last step writes all the counts, explicitly casting both student count and teacher count to the standard IntegerType() columns. It appends the data to the database directly using a Java Database Connectivity (JDBC) connection.

```
facts_data_casted.write.jdbc(
    **POSTGRES_CONFIG,
    table=FACT_TABLE_NAME,
    mode="append"
)
```

5.0 Evaluation

5.1 Pipeline Performance and Efficiency Metrics

Three quantitative dimensions, namely execution latency, data quality retention, and infrastructure throughput were used to benchmark the success of the developed Apache Spark ETL pipeline. The following is a summary of the metrics that were captured from the production execution run:

Evaluation Metric	Captured Value	Technical & Business Meaning
Total Ingestion Volume	2,792,984 rows	The scale of the longitudinal Brazilian Basic Education Census data for 12 years is shown as "Total Ingestion Volume."
Data Quality Retention	100% (0 rows dropped)	Ensures schema purity and absolute data integrity in the data transformation process.
Total Pipeline Latency	418.38 seconds (6.97 mins)	Complete Time Taken to Extract Raw CSVs, Convert to Parquet, Map Star Schema Dimensions and Append to PostgreSQL
Raw Storage Volume	2,540.97 MB (~2.54 GB)	The initial storage footprint of the infrastructure that uncompressed, comma-separated physical files require.
Optimized Parquet Size	322.25 MB	Final compressed storage footprint after Parquet conversion
Storage Compression	87.3% reduction	Significant disk space saving achieved through compression
Throughput Efficiency	~6,677 records/sec	The throughput efficiency of the local Spark cluster with a 4 core standalone parallel architecture

5.2 Interpretation Linked to Business Objectives

The main business objective of this project was to develop a scalable data architecture that processes huge, unstructured public education data into a clean and query optimized Star Schema in PostgreSQL. The indicators reflect that this goal has been met. The processing of 2.79M rows in less than 7 minutes shows how Apache Spark can be scaled for large-scale operations which would otherwise crash or run indefinitely with the likes of spreadsheet software or the standard relational engines. The 2.54 GB to 322.25 MB storage reduction of 87.3% CSV to Parquet conversion is especially noteworthy since downstream queries will be running on a much smaller data portion, directly enhancing the analytical query speed. As there are no rows lost in the whole pipeline, then with full confidence on the underlying data integrity, business analysts can conduct high fidelity, multi-dimensional queries on school facilities such as correlating Internet availability to teacher distribution by state.

5.3 Identified System Limitations

Although the pipeline run was successful, three significant limitations were found during the objective engineering evaluation.The first is Parquet write performance. It took 37.96 seconds to write 100,267 rows to Parquet locally on a Windows environment. For the current dataset size, this is not an issue but would be a significant bottleneck when scaling to larger datasets and/or more runs of the pipeline, as the conversion to Parquet, without the other steps, took about 2 minutes of the total execution time.

The second drawback is JDBC write-bottleneck. The most time was taken was in appending to the local PostgreSQL database instance using JDBC. Relational databases receive data streams sequentially in transactional blocks and this makes the architecture a bottleneck if the data volume is in the millions. This is a natural constraint of the traditional relational database as the load target of a big data pipeline.The third constraint is batch processing constraints. The existing pipeline is a batch process executed at one time. If Brazil Ministry of Education modifies school information at any time during the year, then the whole 6.97-minute pipeline needs to be rerun starting at the beginning. Downstream dashboards and reports would not be updated in real-time, narrowing the scope of dashboards for time-sensitive analytical scenarios.

5.4 Proposed Future Improvements

For scaling to an enterprise-class platform, a number of architectural enhancements are suggested. First, by moving to a cloud solution (like Azure Databricks or Azure Synapse Analytics), these platforms would eliminate Windows compatibility issues that were seen in this project and offer managed Spark clusters, which would sidestep the need to do Hadoop configuration manually. A medallion architecture (bronze: raw CSV data, silver: cleaned and validated Parquet data, gold: final star schema data) could significantly enhance data lineage, data auditing and rollback functionality.
Secondly, switching to a cloud data warehouse (CDW) like Snowflake or Azure Synapse SQL Pool would remove the JDBC write bottleneck due to native columnar storage and massively parallel loading, saving time during the load phase.
Finally, by using automated data quality gateways like Great Expectations the pipeline could automatically detect, flag, and quarantine schema anomalies or corrupted data instead of simply blindly taking all incoming data, ensuring greater data reliability and trustworthiness for business decision-making.

6.0 Reflection

Lim Yu Han

This Apache Spark data pipeline was a good hands-on experience with the challenges of establishing a local data engineering environment. With the infrastructure, running a Linux-focused tool like Spark on Windows had a number of immediate challenges, including Parquet write failures due to the lack of native Hadoop binaries. To correct this, manual path configurations were set to attach the environment to a local Hadoop binary folder. Another cause of network routing issues was a socket conflict with the Docker container for port 5432 with the pre-existing local database. The problem was solved by changing the script configuration to forward traffic to port 5433. In addition, initial resource limits that gave half of the system memory to the driver and half to the executor resulted in freezing when running heavy left joins, which was addressed by dividing the system memory into a 10GB driver and a 6GB executor, providing Spark with sufficient processing power. These challenges were managed effectively and learning outcomes achieved through collaborative technical alignment. Each member of the team did the entire tutorial independently first. The team then discussed the errors they had made in the particular pipeline and the troubleshooting measures taken to correct them, making sure that each member understood the complete operations of the pipeline before splitting the work up strictly for the final report writing. Overall this tutorial was helpful for developing basic data engineering capabilities with PySpark, including how to use PySpark loops to automate table deduplication, and how column-oriented Parquet storage is faster for analytical queries than raw text files.

Ling Yu Qian

The project was quite difficult but also quite rewarding, as it was about creating a real-world ETL pipeline with Apache Spark. The most difficult aspect was to set up the environment on Windows, as the tutorial was aimed at Linux. The download script needed to be rewritten using the requests, zipfile, and shutil libraries from Python, and we have also seen an SSL certificate error when trying to download the source data and the missing PostgreSQL JDBC jar that needed to be downloaded manually. The difficulty of performance tuning was another factor. We initially experienced latency problems, but disabling inferSchema and increasing memory allocation in our 40GB RAM solution resolved that. The Parquet conversion, however, was dropped, as the necessary Windows Hadoop native libraries were not found and reported an incorrect 100% compression ratio. The pipeline was able to perform the complete Parquet conversion, with a real compression ratio of 87.3%, from 2.54 GB to 322.25 MB. It was essential to work as a team throughout. We actively communicated and shared configurations and solutions in real time. One member noticed that disabling inferSchema would greatly speed up the processing time, so they shared it with the group and saved everyone a lot of work. To sum up, in this project, we had hands-on experience with Apache Spark, ETL pipeline design, star schema modeling, Docker, and PostgreSQL. In the future, we would set up the environment earlier, build a Linux virtual machine from the beginning, and implement the proper error handling in the pipeline to make future projects easier to debug.

Teh Ru Qian

This project was challenging because it required an ETL pipeline using Python, Apache Spark, Docker, PostgreSQL and Metabase. Setting up the environment was one of the most challenging tasks because the tutorial mainly used Linux commands. Learning how to use WSL terminal commands and understanding the basics of Linux functions were required for this. Additionally, a few dependencies had to be manually installed, such as installing the missing PostgreSQL JDBC driver and configuring Apache Spark. As team members discussed mistakes, tested fixes and assisted one another in troubleshooting difficulties more effectively, group discussion was crucial in resolving these challenges. Overall, this project improved technical knowledge, problem-solving skills, teamwork, and gave hands-on experience with real-world data engineering operations.